

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	D.HUSSAINIAH	Reg. No.:	192425300
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis**Code of Conduct**

I D.HUSSAINIAH(192425300) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: D.HUSSAINIAH

Huffman Coding vs Fixed-Length Coding: A Comparative Study in Data Compression

Student Name	DUDEKULA HUSSAINAIAH
Student ID	192425300
Assigned Question	Q10 – Huffman Coding vs Fixed-Length Coding
Comparison Type	Greedy (Huffman) vs Static / Fixed-Length Encoding
Domain	Data Compression / Information Theory

1. Objective

Given the symbol frequencies $\{A:5, B:9, C:12, D:13, E:16, F:45\}$, this analysis constructs a Huffman code (Greedy method) and a Fixed-Length code for the same alphabet, compares them on compression efficiency and code length, and evaluates the strengths, limitations, and real-world applicability of each.

2. Assigned Problem Statement

Given symbol frequencies $\{A:5, B:9, C:12, D:13, E:16, F:45\}$, construct Huffman codes and compare with fixed-length encoding. Analyze compression efficiency and code length. Critically evaluate advantages and limitations. Generate insights on real-world data compression.

3. Fixed-Length Coding — Method

With 6 distinct symbols, a fixed-length code needs $\lceil \log_2 6 \rceil = 3$ bits per symbol so that every symbol has a unique bit pattern ($2^3 = 8 \geq 6$). Every symbol — whether it occurs 45 times or 5 times — is charged exactly the same 3 bits. Codes can simply be assigned by index: F=000, E=001, D=010, C=011, B=100, A=101.

4. Huffman Coding — Step-by-Step Construction

Huffman coding is a Greedy algorithm: at every step it merges the two lowest-frequency nodes into a new node, guaranteeing that the least-frequent symbols end up deepest (longest codes) and the most-frequent symbol ends up shallowest (shortest code).

Step	Two Smallest Nodes Merged	New Combined Node	Remaining Active Nodes
1	A(5) + B(9)	AB(14)	AB(14), C(12), D(13), E(16), F(45)
2	C(12) + D(13)	CD(25)	AB(14), CD(25), E(16), F(45)
3	AB(14) + E(16)	ABE(30)	CD(25), ABE(30), F(45)
4	CD(25) + ABE(30)	CDABE(55)	F(45), CDABE(55)
5	F(45) + CDABE(55)	ROOT(100)	ROOT(100) — tree complete

Resulting binary tree (0 = left branch, 1 = right branch):

ROOT(100)

├─ 0 → F(45)

└─ 1 → CDABE(55)

 ├─ 0 → CD(25)

 ├─ 0 → C(12) → code 100

 └─ 1 → D(13) → code 101

 └─ 1 → ABE(30)

 ├─ 0 → AB(14)

 ├─ 0 → A(5) → code 1100

 └─ 1 → B(9) → code 1101

 └─ 1 → E(16) → code 111

5. Code Table and Bit Length Comparison

Symbol	Frequency	Probability	Fixed-Length Code (3-bit)	Huffman Code	Huffman Code Length
F	45	0.45	000	0	1 bit
E	16	0.16	001	111	3 bits
D	13	0.13	010	101	3 bits
C	12	0.12	011	100	3 bits
B	9	0.09	100	1101	4 bits
A	5	0.05	101	1100	4 bits

6. Compression Efficiency — Worked Calculation

Fixed-length total size = 100 symbols $\times$ 3 bits = 300 bits.

Huffman total size = Σ (frequency $\times$ code length) = $(45 \times 1) + (16 \times 3) + (13 \times 3) + (12 \times 3) + (9 \times 4) + (5 \times 4)$
 $= 45 + 48 + 39 + 36 + 36 + 20 = 224$ bits.

Average code length: Fixed-length = $300 / 100 = 3.00$ bits/symbol. Huffman = $224 / 100 = 2.24$ bits/symbol.

Space saved by Huffman coding = $(300 - 224) / 300 \times 100\% \approx 25.3\%$.

7. Head-to-Head Comparison

Metric	Fixed-Length Coding	Huffman Coding (Greedy)
Bits per symbol	Constant — 3 bits for every symbol ($\lceil \log_2 6 \rceil$)	Variable — 1 to 4 bits, driven by frequency
Total encoded size (100 symbols)	300 bits	224 bits
Average bits / symbol	3.00	2.24
Compression achieved vs fixed-length	Baseline (0%)	$\approx 25.3\%$ smaller
Frequent symbols (e.g. F, 45%)	Same 3 bits as everyone else — wasteful	Just 1 bit — highly efficient
Rare symbols (e.g. A, 5%)	Same 3 bits — no penalty	4 bits — slightly longer, acceptable trade-off
Decoding approach	Fixed window — read exactly 3 bits at a time	Prefix-free tree traversal — no delimiters needed
Construction complexity	$O(1)$ — just assign codes by index	$O(n \log n)$ — priority-queue-driven tree build
Adaptivity to data	None — same code table regardless of frequency	Fully adaptive — code table is built from the actual data

8. Why Greedy Works Optimally Here

- Greedy-choice property: always merging the two least-frequent nodes first never produces a worse tree than any other merge order, because a lower-frequency symbol can always be pushed one level deeper without increasing total weighted cost more than swapping it with a higher-frequency symbol would.
- Optimal substructure: once the two smallest nodes are merged into a single combined node, the rest of the problem is identical in shape to the original problem with one fewer node — so an optimal tree over n symbols is built from an optimal tree over $n - 1$ symbols.

- Because both properties hold, Huffman coding is provably optimal among all prefix-free (uniquely decodable) codes for a given frequency distribution — no other prefix code can achieve a lower average bit length.

9. Correctness and Decodability

Both codes are validated as correct and unambiguous:

- Fixed-length code: trivially unambiguous — the decoder simply reads 3 bits at a time.
- Huffman code: satisfies the prefix property (no codeword is a prefix of another — e.g. 0, 100, 101, 111, 1100, 1101 — so a decoder can walk the tree bit-by-bit and never faces ambiguity about where one codeword ends and the next begins, without needing any delimiter characters.

10. Complexity Analysis

Operation	Fixed-Length Coding	Huffman Coding
Building the code table	$O(1)$	$O(n \log n)$ — n inserts/extracts on a min-heap
Encoding the full message	$O(m)$ — m = message length	$O(m)$ — one table lookup per symbol
Decoding the full message	$O(m)$	$O(m)$ — one tree-walk per symbol
Space for code table	$O(n)$ — n = alphabet size	$O(n)$ — tree with n leaves, $n - 1$ internal nodes

Huffman coding pays a one-time $O(n \log n)$ cost to build the optimal code table (n = alphabet size, using a min-heap/priority queue), which is negligible compared to the size of real data streams. Fixed-length coding has no construction cost at all but permanently pays for that simplicity in every encoded message through a higher average bit rate.

11. Real-Time / Real-World Industry Applications

Both approaches map directly onto compression systems used in production today:

- ZIP / GZIP / PNG (DEFLATE algorithm): uses Huffman coding as its final entropy-coding stage after LZ77 pattern matching, exactly like this exercise — frequent byte values get short codes, rare ones get long codes.
- JPEG image compression: after DCT and quantization, JPEG entropy-codes the resulting coefficients with Huffman tables tuned to typical image statistics, which is why bright/flat image regions compress far better than noisy ones.
- MP3 / AAC audio codecs: apply Huffman coding to quantized frequency-domain audio coefficients as the final lossless compression step.
- HTTP/2 header compression (HPACK): uses a static Huffman table (RFC 7541) to shrink repetitive HTTP header text like 'Content-Type' and 'text/html' before transmission.
- Fax machines (ITU-T Group 3/4 standard) and early modems: used run-length + Huffman coding to compress mostly-white scanned pages, which is functionally the same 'skewed-frequency' scenario as this problem (a small number of symbols dominate).
- Fixed-length coding in practice: ASCII (7-bit) and UTF-32 are fixed-length text encodings — simple and universally interoperable, but they are exactly why UTF-8 (a variable-length, Huffman-inspired design) is preferred for real internet traffic, where English text is dominated by a small set of common characters.

12. Strengths and Limitations

- Fixed-length strength: extremely simple, constant-time random access to the k-th symbol, no code table needs to be transmitted alongside the data.
- Fixed-length limitation: ignores symbol frequency entirely, so it wastes bits on skewed real-world distributions (text, images, audio all have skewed symbol frequencies).
- Huffman strength: provably optimal average code length among prefix codes; adapts automatically to the actual data distribution.
- Huffman limitation: variable-length codes break random access (you must decode from the start to reach symbol k); the code table itself must be stored or transmitted, which adds overhead for very small inputs; and it only reaches the true entropy bound when symbol probabilities are powers of 2 — otherwise arithmetic/range coding can do marginally better.

13. Insights on Method Selection

- Choose Huffman (or similar entropy coding) whenever symbol frequencies are skewed and the data is large enough that the one-time cost of building/storing a code table is dwarfed by the savings across the message — this is the common case for text, image, audio, and log-file compression.
- Choose fixed-length coding when the alphabet is already close to uniformly distributed, when constant-time random access into the encoded stream matters more than size, or when the messages are so short that a Huffman table's overhead would outweigh its savings.
- In production compression pipelines, the two are often combined in stages: a pattern-matching transform (like LZ77) first turns the data into a more skewed symbol distribution, and Huffman coding is then applied as the final entropy-coding layer — exactly the design used inside ZIP, PNG, and JPEG.

14. Conclusion

For this symbol set, Huffman coding reduces the encoded size from 300 bits to 224 bits — a 25.3% saving over fixed-length coding — by exploiting the fact that F alone accounts for 45% of all occurrences. This result generalizes: the more skewed a data source's symbol frequencies are, the greater Huffman coding's advantage over fixed-length coding, which is precisely why every major compression format in production use today builds on this same greedy, frequency-driven idea.